基于差分进化模型的烟幕干扰弹的投放策略

摘 要

本文针对来袭导弹的烟幕干扰问题，建立以无人机飞行参数和烟幕弹投放时间为决策变量的差分进化优化模型，并求解最优投放策略。

针对问题一，在题目所给的多种条件下，建立了单枚烟幕弹对来袭导弹的遮蔽效能评估模型。模型基于平抛运动和球—线段几何判断，考虑无人机投放位置、烟幕下沉速度和导弹轨迹，采用时间步长为 0.01s 的离散迭代方法，对导弹—真目标连线与烟幕球的交互情况进行逐时检测，并累计所得的遮蔽时长。计算结果显示，单枚烟幕弹在全局时间 8.04s 到 9.44s 之间形成连续遮蔽，总共的有效遮蔽时长为 1.41s。

针对问题二，研究无人机 FY1 投放单枚烟幕弹的最优策略。建立多种决策变量的差分进化（DE）优化模型，目标是最大化导弹—真目标连线的累计遮蔽时间。约束包括速度范围、航向限制以及投放和起爆的时间关系。结果表明，最优方案下无人机航向角约为44.279°、速度为 140m/s，烟幕在起爆后形成的有效遮蔽累计时长达到 4.84s，相比于问题一中固定策略的 1.41s 有明显提升。

针对问题三，在单枚烟幕优化基础上，研究无人机 FY1 投放三枚烟幕弹的联合策略。以多维因素为决策变量，建立差分进化模型，目标为最大化的总计并集遮蔽时间。结果表明，最优方案下无人机航向角为7.770°、速度为 105.112m/s，三枚烟幕依次起爆形成连续遮蔽，总遮蔽时间提升至 6.41s，较单枚方案明显增加。

针对问题四，对导弹来袭，建立多无人机协同投放烟幕的优化模型，以多维因素为决策变量，采用几何判据与时间离散计算遮蔽时长，并用差分进化算法求解。求得结果，三架无人机的最优策略使得总遮蔽时长达到 14.85s。显著优于单机投放或者少量烟幕的情形。

关键词：离散模型，差分进化算法，遮蔽时间优化，多弹协同投放

一、 问题重述

1.1 问题背景

在现代的战争环境中，空地导弹对固定目标的精确打击能力在不断增强，传统的目标伪装和防护手段已经难以发挥有效作用。作为一种较为重要的软杀伤防护方式，烟幕干扰技术能够在目标与来袭武器之间形成遮蔽，削弱或阻断攻击导弹的探测与制导过程，从而有效地提高目标的生存概率。烟幕干扰弹通常会通过化学燃烧或者爆炸分散形成气溶胶烟幕弹，并依靠其在空间的弥散和下沉运动构建有效的遮挡区域，具有成本低、使用灵活、效率比高等优点。

随着无人机技术的发展，利用无人机精确投放烟幕干扰弹成为实现精确、快速防护的重要手段。无人机可根据任务需求调整飞行方向和速度，并在合适的时机与位置投放烟幕干扰弹，使其在真目标前方形成有效的遮蔽烟幕弹。投放过程需要综合考虑导弹的飞行轨迹、目标位置、烟幕烟幕弹的运动特性以及有效的作用时间，以便确保在导弹临近目标前提供尽可能长的遮蔽效果。

综上，如何去建立数学模型并设计无人机的投放策略，合理安排其飞行路径、投放点和起爆时序，使得烟幕干扰弹在有限的数量下发挥最佳防护效果，是本问题研究的核心。

1.2 问题提出

（1）在给定 FY1 部分参数的情况下，计算对 M1 的有效遮蔽时长；

（2）自由优化 FY1 的飞行方向、速度、投放点、起爆点，求对 M1 单枚导弹的最大可能遮蔽时长；

（3）利用一架无人机（FY1）投放 3 枚干扰弹去干扰同一目标（M1），给出优化策略；

（4）利用三架无人机（FY1-FY3）各投 1 枚弹干扰同一目标（M1），给出优化策略；

（5）利用五架无人机（每架最多 3 枚弹）干扰三个目标（M1，M2，M3），给出优化策略。

二、 问题分析

2.1 概论

在现代战场中，精确制导弹武器的广泛应用对重要军事目标构成严重威胁。

然而烟幕干扰技术作为一种成本低、效费比高的电子对抗手段，能够在目标前方特定空域形成遮蔽，干扰敌方导弹。烟幕干扰弹主要通过化学燃烧或爆炸分散形成烟幕或气溶胶烟幕弹，在特定的空域中形成遮蔽，降低导弹的命中概率。随着无人机技术的不断发展，用无人机投放烟幕干扰弹成为一种十分高效的战术选择。然而，如何优化无人机投放策略以最大化烟幕遮蔽，仍然是一个涉及多目标优化和时空协同的复杂问题。

2.2 问题一

单架无人机干扰单枚来袭导弹时，需要综合无人机初始位置、高度、航向与速度、飞到投放点所需时间，以及投放后弹体的平抛运动和引信延时。起爆瞬间形成的烟幕在 20s 内、半径 10m 范围内有效遮蔽，因此关键判断是起爆时间与起爆时间与起爆位置相对于真目标及导弹到达时刻的时空关系。数值上采用时间离散化[1]，在每一时刻用几何判据检测导弹—真目标连线是否与烟幕球相交以累计遮蔽时长。求解时还必须满足飞行与安全约束，目标是界定可行的干扰时间并评估实施可行性，为投放策略提供决策依据。

2.3 问题二

在问题二中，目的在于通过优化无人机飞行方向、速度、投放点及烟幕弹引信时间，延长对单枚导弹的遮蔽持续时间。本问题的分析需综合考虑无人机的运动学限制、烟幕弹的持续有效时间和下沉速度以及烟幕弹的水平和垂直位移。无人机水平速度决定了其到达目标上方的最早时间，引信时长受重力限制；烟幕弹的有效遮蔽时间限制了可用的干扰窗口。通过对这些因素进行共同分析，可以确定无人机飞行路径与投放时序，使烟幕干扰弹爆炸时尽量接近目标上空，并在导弹抵达前提供最长的遮蔽时间，为建立最优模型奠定基础。

2.4 问题三

在问题三中，研究单架无人机投放多枚烟幕干扰弹的策略。分析过程需要关注连续投放带来的遮蔽时间的积累效应，同时满足投放间隔不小于 1 秒钟的约束。每枚烟幕干扰弹爆炸形成的烟幕弹持续有效约 20 秒，投放位置和引信时长会影响烟幕弹对真目标的覆盖效果。通过计算每枚弹的爆炸位置与目标的空间关系，可以评估覆盖的连续性与遮蔽的总时长。此外，还需要考虑无人机飞行速度、路径以及初始位置对多枚投放的协调性。本问题的分析目的是明确如何利用有限的烟幕干扰弹实现时间和空间上的最优遮蔽策略。

2.5 问题四

在问题四中，涉及多架无人机协同投放干扰弹对单枚导弹的干扰策略。分析中需要考虑无人机之间的相对位置、飞行速度和投放协调，以最大化烟幕烟幕弹对目标的覆盖时间。每架无人机的投放时序与位置可直接影响烟幕弹的叠加效果与遮蔽连续性，同时需要保证无人机之间投放间隔不冲突。通过分析不同无人机在空间与时间上的协同分布[2]，可以优化任务分配，使多架无人机形成连续或重叠的遮蔽区域，从而可以显著的提升整体的防护效率。此问题的核心在于明确协同规则和遮蔽优化原则，可为后续多架无人机投放模型提供依据。

2.6 问题五

在问题五中，将问题扩展到了多架无人机的同时对多枚导弹实施干扰的情境。分析需综合考虑每枚导弹的飞行轨迹、到达时间和各无人机可控的投放参数，包括位置、速度以及引信时长。同时，需要合理分配无人机资源，使有限数量的干扰弹在时间和空间上有效的覆盖所有来袭导弹，保证遮蔽时间的最大化。还需要考虑烟幕烟幕弹可能的重叠效应和多目标干扰的优先级，以及策略的实用性。通过对这些因素的分析，可以指导建立多无人机多目标投放模型，实现复杂战场条件下的最优防护布局。

三、 模型假设

1.假设无人机在投放的过程中沿直线飞行，不考虑航向修正或者加减速；

2.假设干扰弹投放后仅受重力作用，水平速度保持恒定，忽略空气受阻力和侧向漂移；

3.假设多枚烟幕干扰弹的烟幕弹遮蔽时间可以不连续，不考虑烟幕弹浓度衰减或者互相干扰；

4.假设导弹沿直线恒速飞行，不考虑制导修正或者环境影响；

5.假设无人机接到任务后可以瞬间调整航向，不考虑转向花费的时间；

6.假设场地平坦、无障碍，风速、气象变化等对烟雾弹和烟幕弹运动影响可以忽略；

7.假设无人机之间及与地面障碍物的安全距离固定，不考虑碰撞的风险。

四、 符号说明

符号	符号说明
$t_{explosion}$	全局起爆时间(s)
c_{down}	烟幕球下降速度(m/s)
c_{cover}	烟幕球有效遮蔽时间(s)
t_{drop}	投放干扰弹时间
$\hat{u}$	单位方向向量
W	导弹到云团的向量
τ	延时引爆时长
c_0	烟幕球球心
R	烟幕球有效半径
s	烟幕球心到导弹—真目标连线的参数投影
s_{clamp}	将 s 裁断到[0,1]
$total_{time}$	总计有效遮蔽时长

五、 模型建立与求解

5.1 问题一

5.1.1 问题一分析

在警戒雷达发现来袭导弹 M1 后，单架无人机 FY1 在受领任务以后以 120m/s 朝假目标方向匀速飞行，1.5s 后投放一枚烟幕干扰弹，投放后 3.6s 起爆。起爆瞬间形成球状的烟幕烟幕弹，烟幕弹在起爆后以 3m/s 的速度匀速下降；烟幕弹中心 10m 范围内在起爆后 20s 内可为真目标提供有效遮蔽。导弹 M1 以 300m/s 匀速直线飞向假目标（原点）。在上述简化假设下，求该枚烟幕弹对导弹 M1 的有效遮蔽时长（导弹在“真目标—导弹当前的位置”连线段与烟幕弹相交所积累计的时间）。

烟幕的遮蔽效果在几何上等价于“在某一时刻t，烟幕弹中心到导弹-真目标连线段的最短距离≤10m”。因此问题变为：随时间t推移，判断球（烟幕弹）与线段（导弹当前位置到真目标）是否相交；满足相交的时间段进行计算累计，便得到有效时间。由于烟幕弹从起爆时刻$t_{explosion}$开始存在并在$t_{explosion}$~$t_{explosion}+20$之间下沉，我们只需要在该时间窗内检查相交情况并求

时间长度。

5.1.2 离散模型的建立

（1）坐标与已知量

参考系：右手笛卡尔坐标系，假目标为原点$fake_{target}$=(0,0,0)。具体位置如图 1 所示：

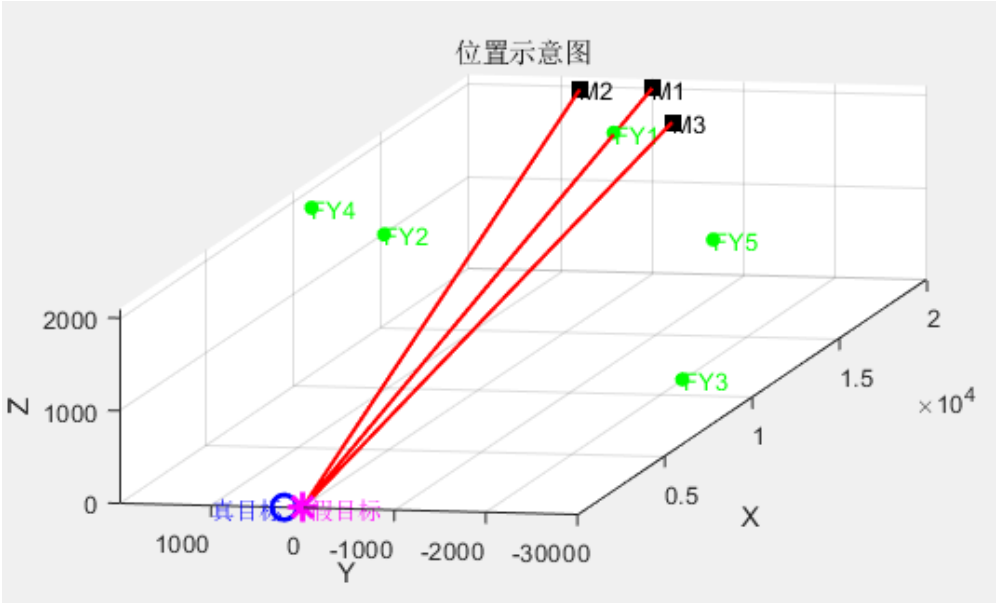


图 1 导弹、无人机、真假目标位置示意图

导弹 M1 初始位置：

$$M_1 = (20000,0,2000)$$

真目标位置：

$$T = (0,200,0)$$

无人机 FY1 初始位置：

$$F_1 = (17800,0,1800)$$

无人机速度：$v_u = 120m/s$。投放时刻为$1.5s$后，无人机 FY1 位置为(17620,0,1800),干扰弹延时起爆$3.6s$，因此起爆全局时间为：

$$t_{explosion} = 5.1s$$

由平抛运动公式可得起爆时刻烟幕球心坐标：

$$C_0 = (17188,0,1736,496)$$

烟幕弹参数：下沉速度$c_{down} = 3m/s$，半径$R = 10m$，持续时间$c_{cover} = 20s$。

导弹速度：$V_m = 300m/s$。设导弹指向假目标，其单位方向向量为：

$$\hat{u} = \frac{(0,0,0) - M_1}{\|(0,0,0) - M_1\|} \tag{1}$$

（2）运动学方程

1)导弹位置：

$$M(t) = M_1 + V_m\hat{u}t \tag{2}$$

2)烟幕弹中心位置：

起爆时刻为$t_{explosion}$，起爆瞬间云心C_0；之后烟幕球匀速下沉：

$$C(t) = C_0\left(0,0,-c_{down}\left(t - t_{explosion}\right)\right), t \in \left[t_{explosion}, t_{explosion} + c_{cover}\right] \tag{3}$$

3)导弹-目标连线段：

在任一时刻t，连线段导弹当前的位置$M(t)$到真目标T。用参数 s∈ [0,1]表示该线段上点：

$$L(s;t) = M(t) + s\left(T - M(t)\right), s \in [0,1] \tag{4}$$

（3）遮蔽条件

在烟幕弹存在的时间窗的 t，若烟幕球与导弹—目标线段相交，则认为该时刻导弹对真目标被烟幕遮蔽。等价判定条件为：球心到线段的最短距离≤R。

具体的计算步骤（每 0.01s）：

1)令

$$v = T - M(t) \tag{5}$$

与

$$W = C(t) - M(t) \tag{6}$$

2)先求线段参数投影：

$$s = \frac{W \cdot V}{V \cdot V} \tag{7}$$

将 s 截到区间[0,1]：

$$s_{clamped} = max(0, min(1, s)) \tag{8}$$

3)线段上最近点为$L(s_{clamped};t)$，最短距离为：

$$d(t) = \left\| C(t) - L\left(s_{clamped}; t\right) \right\| \tag{9}$$

4)若$d(t) \geq R$，则时刻 t 被计入遮蔽时间；否则不计入。

5.1.3 **模型的求解**

本文采用精确运动学表达+高精度时域检测的数值求解方法：在烟幕弹存在的时间窗$[t_{explosion}, t_{explosion} + c_{cover}]$内对时间进行细化（时间步长$\Delta t = 0.01s$），对每个时间点执行上节的遮蔽判定并累积遮蔽时间。

（1）数值求解流程

1)设置常量：

$V_m = 300m/s$、$c_{down} = 3m/s$、$R = 10m$等；

2)时间离散：

$t_k = t_{explosion} + k\Delta t(k = 0,1, \ldots, N)$,其中$\Delta t = 0.01s$，$N = \lfloor 20/\Delta t \rfloor$；

3)对每个时间点t_k：

① 计算$M(t_k)$、$C(t_k)$、向量 V，W，计算 s、截断为$s_{clamped}$，计算最短距离$d(t_k)$；

② 若$d(t_k) \leq R$，则把该步时长Δt累加到$total_{time}$，并记录开始/结束时间段以输出连续区间。

4)最终$total_{time}$为累计遮蔽时长，连续相交区域通过首尾 t 记录为若干区间。

注:时间步长$\Delta t = 0.01$足够小，能保证数值误差极小(约等于 0.01s 量级)。

（2）关键数值中间量

导弹单位向量$\hat{u}$从M_1指向原点计算得：

$$\hat{u} = \frac{(-20000,0,-2000)}{\sqrt{20000^2 + 2000^2}} \approx (-0.99501115706,0,-0.099501115706) \tag{10}$$

导弹速度矢量：

$$v_m = V_m\hat{u} \approx \frac{(-298.51115706,0,-29,8501115706)m}{s} \tag{11}$$

起爆时刻与起爆点：

$$t_{explosion} = 5.1s, C_0 = (17188,0,1736.496) \tag{12}$$

按照上述数值在时间窗$[5.10s, 25.10s]$用$\Delta t = 0.01s$逐点执行线段-球体相交判定；

检测的时间段为：5.10s 到 25.10s（步长 0.01s）；

最终得到累计 遮蔽时长$total_{time}$=1.4100s。

其中检测到的连续遮蔽区间（时间以全局时刻计）为：从 8.04s 到 9.44s。

5.2 问题二

5.2.1 问题二分析

在问题 1 的情形下，现在要对无人机 FY1 的飞行方向、速度、烟幕弹的投放点和起爆点进行考虑选择。通过选择这些因素，从而使得单枚烟幕干扰弹在起爆及其之后的 20s 内，对来袭导弹 M1 与真目标之间的线段的累计有效遮蔽时间$total_{time}$最大化。问题主要在于无人机运动于干扰弹起爆时间的协调，确保烟幕弹在导弹经过与真目标的连线上从而提高遮蔽时间。

5.2.2 差分算法模型的建立

（1）决策变量

定义优化问题的决策表变量为：

1)无人机航向角θ：水平面内无人机飞行方向，单位：rad。

2)无人机速度v_u：无人机匀速飞行速度，范围：70-140m/s。

3)烟幕起爆全局时刻$t_{explosion}$：干扰起爆时刻，决定烟幕弹进入遮蔽区域的时间窗口。

4)引信延时τ：投放到起爆的时间间隔，投放时刻为$t_{drop} = t_{explosion} - \tau$。

综合决策向量为：

$$\boldsymbol{P} = (\theta, v_u, t_{explosion}, \tau)$$

（2）目标函数

目标函数为最大化单枚干扰弹在导弹—真目标连线段的遮蔽时间：

$$total_{time}(\boldsymbol{P}) = \sum_{t=t_{explosion}}^{t_{explosion}+20} \mathbf{1}\{d(t;\boldsymbol{P}) \le R\} \cdot \Delta t \tag{13}$$

其中：

$d(t;\mathbf{p})$为导弹—真目标线段与烟幕弹球心的最短距离，用线段投影公式计算：

$$s = \frac{(\boldsymbol{C}(t) - \boldsymbol{M}(t)) \cdot (\boldsymbol{T} - \boldsymbol{M}(t))}{\|\boldsymbol{T} - \boldsymbol{M}(t)\|^2}，\ s_{clamp} = min\big(1, max(0, s)\big) \tag{14}$$

$$L = M(t) + s_{clamp}\big(\boldsymbol{T} - \boldsymbol{M}(T)\big), d(t) = \|\boldsymbol{C}(t) - \boldsymbol{L}\| \tag{15}$$

烟幕弹位置随时间下沉：

$$\boldsymbol{C}(t) = \boldsymbol{C_0} + [0,0,-c_{down}(t - t_{explosion})] \tag{16}$$

$R=10m$ 为烟幕弹有效遮蔽半径，Δt为时间离散步长。

（3）约束条件

1)无人机速度约束：

$$70 \le v_u \le 140m/s$$

2)航向角约束：

$$0 \le \theta \le 2\pi$$

3)起爆时间与引信延时约束：

$$t_{explosion} \ge 0,0 \le \tau \le t_{explosion}$$

4)投放时刻约束：

$$t_{drop} = t_{explosion} - \tau \ge 0$$

5.2.3 差分算法模型的求解

（1）算法实现

1)差分算法[3]（DE）是一种群体启发式全局优化算法，适合连续参数优化：

初始化种群：随机生成N_p个候选解p_i在约束范围内。

2)变异操作：生成变异向量

$$v_i = p_{r1} + F \cdot (p_{r2} - p_{r3}) \tag{17}$$

其中$r1, r2, r3$为随机不同个体索引，F为缩放系数。

3)交叉操作：生成试验个体u_i

4)选择操作：若$total_{time}(u_i) > total_{time}(p_i)$则替换。

5)迭代至收敛或达到最大迭代次数。

计算的时间步$\Delta t = 0.01s$，群体大小 100，迭代 500 次。迭代图如图 2 所示：

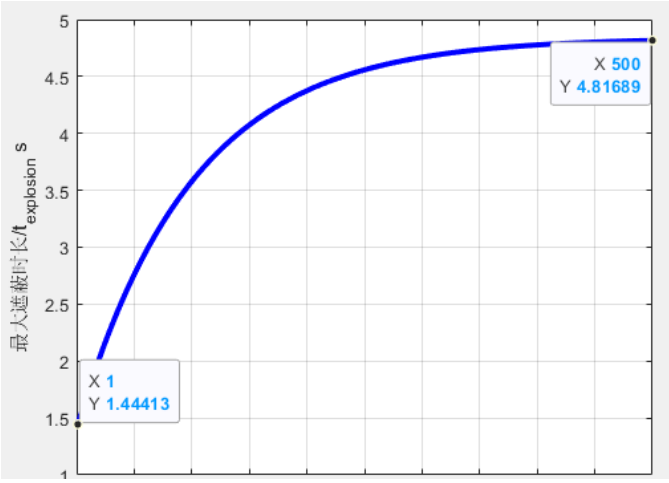


图 2 差分进化算法收敛曲线

对最优候选解采用Δt=0.01s 精细计算遮蔽区间，得到累计遮蔽时长、进入和离开的时间。

速度和航向角超界则截断至允许的范围；起爆时间与引信延时约束在生成时直接保证。

（2）计算结果

经过差分进化优化与数值评估，得到单架无人机投放单枚烟幕干扰弹的最优策略如表 1 所示：

表 1 问题 2 多变量最优解

决策变量	**最优解**
无人机航向角θ	7.056rad（≈44.279°）
无人机速度v_u(m/s)	140.000
投放时刻t_{drop}(s)	0.000
引信延时τ(s)	0.708
起爆时刻$t_{explosion} = t_{drop} + \tau$(s)	0.708
起爆坐标	[17898.309, 12.168, 1797.547]
累计遮蔽时长$total_{time}$(s)	4.840

累计遮蔽时长$total_{time} \approx 4.84s$。起爆坐标以及遮蔽时间如图 4 和图 3 所示：

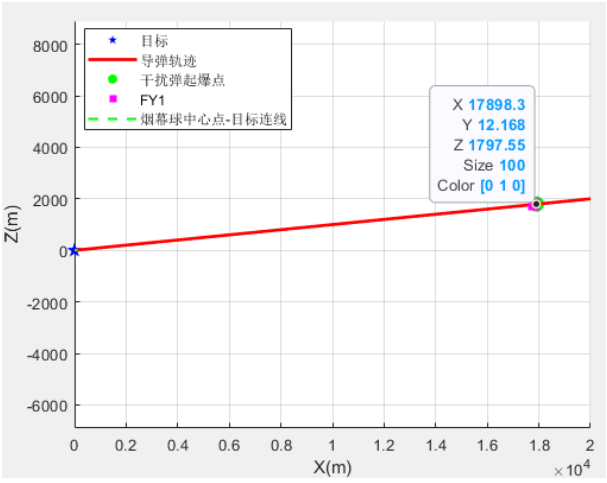


图 4 无人机-目标-导弹-烟幕弹空间示意图

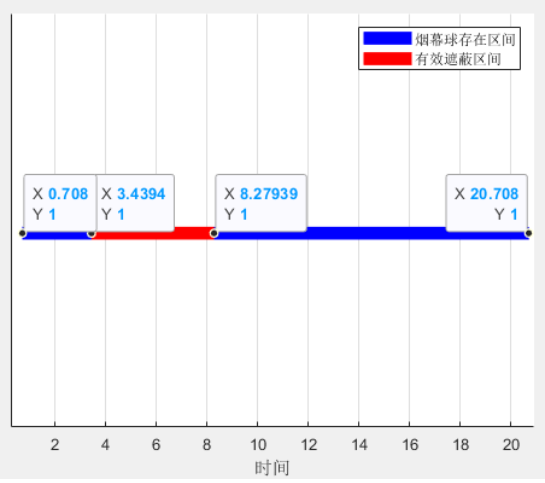


图 3 无人机烟幕遮蔽时间区间

（3）结果分析

与问题 1 中固定策略所得的 1.41s 相比，优化策略显著提升了遮蔽效果。由此可以看出，通过合理选择无人机的飞行方向、速度、烟幕弹的投放点和起爆点[4]，可以在有限的条件下实现更长时间的目标掩护。具体时长对比如图 5 所示：

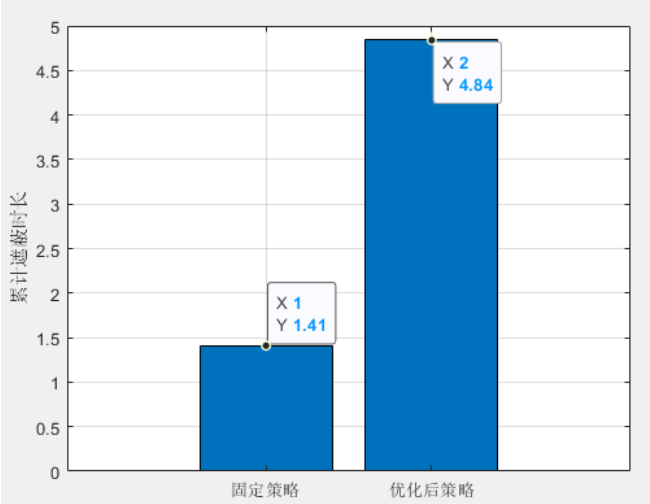


图 5 遮蔽时长对比

5.3 问题三

5.3.1 问题三分析

在问题二的基础上，考虑无人机 FY1 投放三枚烟幕干扰弹，以对来袭导弹 M1 实施干扰。由于三枚干扰弹的投放时刻、引信延时以及起爆时间可以不同，需要协调三枚烟幕烟幕弹在导弹—真目标连线上形成尽可能连续的遮蔽效果，从而提高总并集遮蔽时间。

问题的主要目标在于合理选择无人机飞行方向、飞行速度以及三枚烟幕干扰的投放和起爆时刻，使得多枚烟幕干扰弹在覆盖导弹路径时遮蔽时间递增、连续且总并集最大。同时每枚烟幕弹的投放间隔必须大于 1 秒，且烟幕弹下沉不致过快落地，以保证烟幕有效覆盖。

5.3.2 差分算法模型的建立

（1）决策变量

对三枚烟幕干扰弹的联合优化，决策向量定义为：

$$\boldsymbol{P} = (\theta, v_u, t_{drop1}, \tau_1, t_{drop2}, \tau_2, t_{drop3}, \tau_3)$$

投放时刻约束为递增：

$$t_{drop2} - t_{drop1} \geq 1s,\ t_{drop3} - t_{drop2} \geq 1s$$

引信延时约束：

$$0.5 \le \tau_i \le 5s$$

（2）目标函数

考最大化三枚烟幕干扰弹的总并集遮蔽时间：

$$total_{time}(\boldsymbol{P}) = \sum_{t} 1\{d(t:\boldsymbol{P}) \le R\} \cdot \Delta t \tag{18}$$

其中$d(t;\boldsymbol{P})$为第i枚烟幕烟幕弹球心与导弹—真目标连线段的最短距离，烟幕弹位置随时间下沉：

$$C_i(t) = C_{0i} + \{0,0,-c_{down}(t - t_{explosion\,i})\} \tag{19}$$

并集遮蔽时间通过合并三个烟幕弹各自遮蔽区间计算，避免简单求和重复计入。

（3）约束条件

1)无人机速度约束：$70 \le v_u \le 140m/s$；

2)航向角约束：$0 \le \theta \le 2\pi$；

3)投放时刻递增与间隔约束：$t_{drop2} - t_{drop1} \ge 1, t_{drop3} - t_{drop2} \ge 1$；

4)引信延时约束：$0.5 \le \tau_i \le 5s$；

5)投放位置在无人机可达到范围内。

5.3.3 差分算法模型的求解

（1）算法实现

采用 DE 算法进行全局优化：

1)初始化种群：随机生成 60 个候选解，在约束范围内确保投放时刻递增；

2)变异：

$$v_i = p_{r1} + F \cdot (p_{r2} - p_{r3}) \tag{20}$$

3)交叉：生成试验个体u_i，保证各变量在约束范围内；

4)选择：若试验个体总并集遮蔽时间大于当前个体，则替换；

5)迭代：500 次选代，记录全局最优个体；

具体算法流程图如图 6 所示：

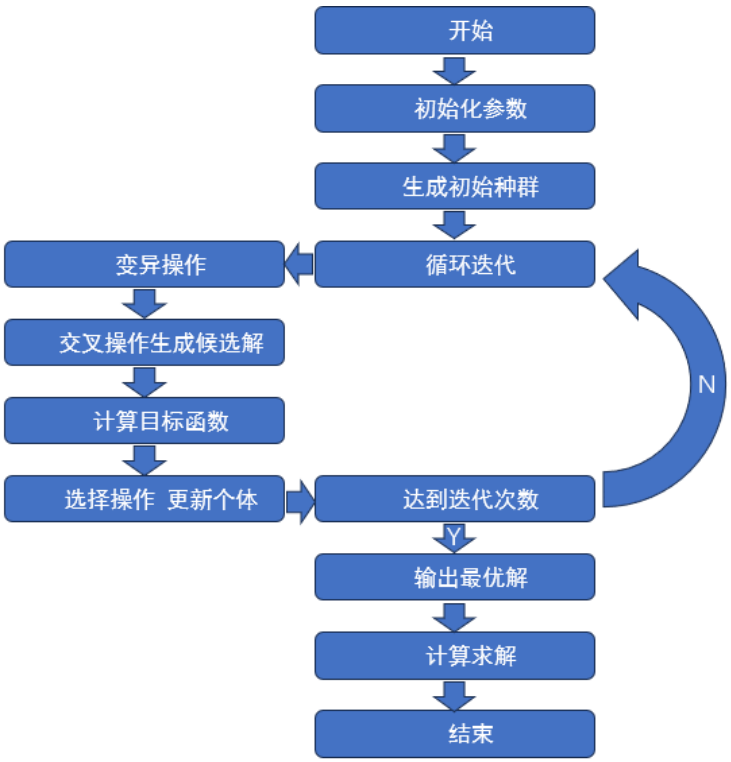


图 6 差分算法流程图

其中：

时间步长：$\Delta t = 0.01s$；

烟幕下沉速度：$c_{down} = 2.5m/s$；

烟幕弹有效半径：$R=10m$；

每枚烟幕有效时间：$c_{cover} = 20s$。

（2）计算结果

经过差分进化优化与精细评估，得到无人机 FY1 投放三枚烟幕干扰弹的最优策略如表 2 所示：

表 2 无人机 FY1 投放策略表

烟幕干扰弹编号	烟幕干扰弹投放点的坐标(m)	烟幕干扰弹起爆点的坐标(m)	有效干扰时长(s)
1	(17800, 0, 1800)	(17800, 0, 1800)	2.64
2	(17904.147, 14.211, 1800)	(17904.147, 14.211, 1800)	4.48
3	(18008.295, 28.422, 1800)	(19643.795, 251.589, 591.626)	0

另外无人机的飞行方向为$\theta = 7.770°$，飞行速度为$v_u = 105.112m/s$。总计遮

蔽时间并集结果为$total_{time} = 6.41s$。

（3）结果分析

与问题二单枚烟幕的遮蔽时长 4.84s 相比，三枚烟幕的总井集遮蔽时间增加至 6.41s，显著提升了干扰效果。

三枚烟幕投放时刻递增，且间隔大于 1s，保证各自有效遮蔽区域在导弹路径上连续衔接。其遮蔽时长效果和迭代寻解如图 7 和图 8 所示：

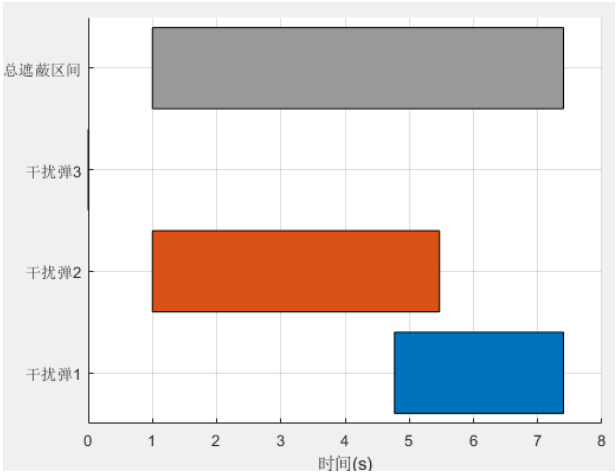


图 7 三枚烟幕弹遮蔽区间示例图

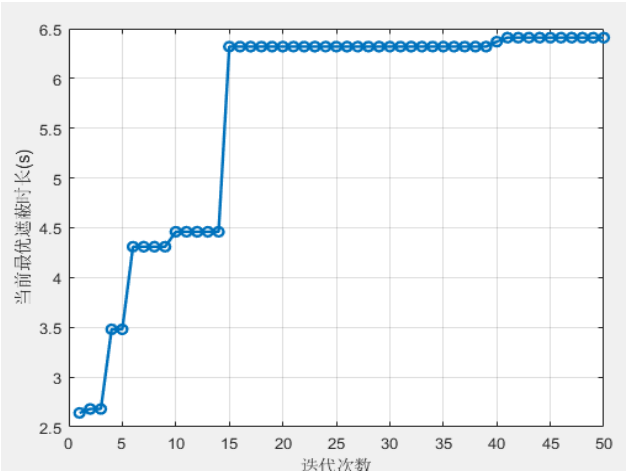


图 8 差分进化算法收敛曲线

烟幕弹下沉速度适中，使烟幕在导弹到达路径时保持高度，防止过早落地。

航向角与速度统一，确保三枚烟幕干扰弹覆盖的空间连续，简化无人机飞行控制。

综上所述，通过合理选择无人机飞行方向、速度及三枚烟幕弹的投放和起爆时刻，能够在有限条件下实现更长时间的目标掩护，总并集遮蔽时间提升明显。

5.4 问题四

5.4.1 问题四分析

为应对多来袭导弹情形，问题四要求利用多架无人机协同投放烟幕干扰弹，使得在导弹来袭过程中真目标被烟幕有效遮蔽的累计时间尽可能长。与前一问相比，本题的难点在于决策变量维数增加（每架无人机的航向、速度、投放时刻与起爆延时），变量间耦合复杂（不同无人机的投放时刻、烟幕球的时序和空间叠加），且遮蔽判定为非光滑、非线性的几何判断，因此采用黑箱全局优化算法[5]进行求解，同时配合物理约束与数值离散策略保证可计算性和可复现性。

5.4.2 差分进化模型建立

（1）符号与参数说明

导弹：初始位置$r_M(0)$,速度标量$v_M = 300m/s$定向假目标。

无人机（第j架）：初始位置$rU_j(0)$,航向单元向量$\mathrm{d}U_j(\theta_j)$,速度vU_j。

烟幕弹（第i枚）：投放时间t_{drop},i、起爆延时τ_i、起爆时刻$t_{expl,i}=t_{drop,i}+\tau_i$、起爆位置$r_{expl,i}$。

烟幕球：半径 R=10m、下沉速度$c_{down}=3m/s$、有效寿命$c_{cover}=20s$。

真目标中心r_T,遮蔽累计时间记为$total_{time}$

（2）运动学模型

1)导弹运动：

$$u_M\frac{r_D-r_M(0)}{\|r_D-r_M(0)\|},r_M(t)=r_M(0)+V_Mu_Mt \tag{21}$$

2)无人机运动：

$$rU_j(t)=rU_j(0)+vU_jd_{U_j}t,d_{U_J}t,d_{U_j}=\left(cos\theta_j,sin\theta_j,0\right) \tag{22}$$

3)烟幕弹运动与起爆点：

且投放点（第i枚对应某架无人机j）：$r_{drop,i}=rU_j(t_{drop,i})$弹体初速度等于无人机速度$vU_jdU_j$。相对投放时刻t′的弹体位置为：

$$r_b(t')=r_{drop,i}+vU_jd_{U_j}t'+\left(0,0,-\frac{1}{2}gt'^2\right) \tag{23}$$

起爆时$\mathrm{t}'=\tau_\mathrm{i}$得到起爆位置$r_{expl,i}=r_b(\tau_i)$。若计算得到$\mathrm{z}<0$，在数值实现中将 z 截断为 0。

4)烟幕球随时间演化：

起爆后在全局时间$t\geq t_{expl,i}$的球心：

$$C_i(t)=r_{expl,i}-\left(0,0,c_{down}\left(t-t_{expl,i}\right)\right),t\in\left[t_{expl,i},t_{expl,i}+c_{cover}\right] \tag{24}$$

（3）遮蔽判定

在任意时刻 t，考虑导弹当前位置$r_M(t)$与真目标中心r_T之间的线段：

$$\mathcal{g}(t):L(s)=r_M(t)+s\left(r_T-r_M(t)\right),s\in[0,1] \tag{25}$$

烟幕球i在时刻 t 的球心为$C_i(t)$。定义投影参数：

$$s^*=\frac{(C_i(t)-r_M(t)\cdot(r_T-r_M(t))}{\|r_T-r_M(t)\|^2} \tag{26}$$

载断到线段区间$s_{cl}=max\left(0,min(1,s^*)\right)$,最短距离：

$$d_i(t) = \left\| C_i(t) - \left(r_M(t) + s_{cl}\left(r_T - r_M(t) \right) \right) \right\| \tag{27}$$

若存在某i使得$\mathrm{d_i(t)} \le \mathrm{R}$,则时刻$t$认为被遮蔽。此几何判据直观且易于数值实现

（4）离散化与目标函数

采用时间离散化$t_k = t_0 + k\Delta t(\Delta t$取 0.01s)，指示函数

$$\delta_k = \begin{cases} 1, & 若\exists i\ s.t, d_i(t_k) \le R, \\ 0, & 否则. \end{cases}$$

总遮蔽时长的离散近似：

$$c_{cover} \approx \sum_k \delta_k \Delta t \tag{28}$$

优化目标为最大化c_{cover}。

（5）约束条件与决策变量集合

假设一共有k架参与的无人机，每架至多投放m枚干扰弹（本题中$k = 3$，$m = 1$）。每架无人机j的决策变量：

$$X_j = \left(\theta_j, vU_j, t_{drop,j,1}, \tau_j, 1, \ldots, t_{drop,j,m}, \tau_{j,m} \right)$$

总体决策向量$X = [X_1, \ldots, X_K]$。变量应满足：

$$v_{min} \le vU_j \le v_{max} \left(70 - \frac{140m}{s} \right) \tag{29}$$

同机相邻投放间隔$\ge 1s$(若 m> 1)；

投放时刻与起爆时间不晚于导弹到达的合理上界；

起爆高度 z≥ 0。

5.4.3 模型求解

（1）采用差分进化[6]求解，参数取$NP = 200$，$G_{max} = 5000$，$F = 0.8$，$CR = 0.9$，时间步长$\Delta t = 0.01s$。每架无人机投放 1 枚烟幕，遮蔽判定采用前文给出的线段—球体几何方法。求得最优策略如表 3 所示：

表 3 问题 4 多机单弹投放策略表

无人机编号	无人机运动方向	无人机运动速度(m/s)	烟幕干扰弹起爆点的 x 坐标 (m)	有效干扰时长(s)
FY1	7.261	123.624	（17893.006，11.850，1797.516）	4.82
FY2	238.240	112.524	（11156.939，38.149，1144.247）	3.35
FY3	151.929	138.631	（-0.057，199.865，26.517）	6.68

（2）结果分析

1)差分进化所得最优方案使三枚烟幕在导弹末端关键时间形成若干有效遮蔽段，累计遮蔽时长明显提升，说明协同投放可显著提高目标隐蔽性。

2)各枚烟幕对c_{cover}的贡献不均，时间/空间重叠会导致遮蔽冗余；合理错开投放时序能提升总体效率，避免资源浪费。

3)对最优解进行小幅扰动测试显示结果对起爆延时比较敏感；建议采用两阶段和并行计算以提高解的稳定性与计算效率。

5.5 问题五

5.5.1 问题五分析

为应对三枚来袭导弹同时攻击目标的情形，本问题要求利用五架无人机协同投放烟幕干扰弹，最大化真目标在导弹来袭过程中被烟幕有效遮蔽的累计时间。每架无人机可以携带三枚烟幕干扰弹，投放时间和起爆时间可以独立设定。本任务的核心是确保无人机飞行方向、速度以及各个烟幕弹的投放点与起爆时刻，使得多枚烟幕弹在导弹路径上尽可能连续地覆盖，避免遮蔽时间空缺或者过度重叠。此问题的主要难点在于：决策变量的维度变高、变量之间耦合复杂、遮蔽判定是非光滑非线性几何问题，同时需要满足无人机速度、航向、投放间隔和烟幕下沉等物理约束。

5.5.2 问题 5 的模型建立

（1）决策变量

记第j架无人机投放的第k枚烟幕干扰弹的决策变量为：

$$X_{j,k} = \{t_{drop}^{j,k}, \tau_b^{j,k}\} \tag{30}$$

其中:

1)$t_{drop}^{j,k}$: 投放时刻；

2)$\tau_b^{j,k}$:起爆延时；

3)无人机速度v_j与飞行方向向量$\overrightarrow{d_j}$在任务开始时选定，并在整个过程中保持不变。

因此整体的决策变量向量为:

$$X = \left\{d_j, v_j t_{drop}^{j,k}, \tau_b^{j,k} | j = 1..5, k = 1..3\right\} \tag{31}$$

（2）烟幕运动与遮蔽判定

1)烟幕球中心位置:

投放后烟幕干扰弹遵循平抛运动，至$\boldsymbol{t_{explosion}} = t_{drop} + \boldsymbol{\tau}_b$时刻起爆，其爆炸点为：

$$P_{explosion}^{j,k} = p_{UAV}^{j}\left(t_{drop}^{j,k}\right) + \Delta P\left(\tau_b^{j,k}\right) \tag{32}$$

其中$\Delta P(\boldsymbol{\tau_b})$由重力决定的竖直位移。

（3）目标函数

定义总遮蔽时长为：

$$f(X)=\sum_{M\in\{M1,M2,M3\}}\sum_{t=0}^{T_{end}}1\left(\text{遮蔽成立}(M,t;X)\right)\cdot\Delta t \tag{33}$$

其中Δt为离散时间步长，$1(\cdot)$为指示函数。

优化目标为：

$$\max_{X} f(X) \tag{34}$$

5.5.3 问题 5 的模型建立

采用差分进化（DE）算法对上述优化问题进行求解，其流程如下：

(1)初始化

随机生成种群个体，每个个体对应一个完整的投放策略。

确保初始解满足约束条件。

(2)适应度计算

对每个个体，根据烟幕球轨迹和导弹轨迹进行遮蔽判定，计算总遮蔽时长。

(3)变异

采用经典 DE/rand/1 方式：

$$V_i=X_{r1}+F\cdot(X_{r2}-X_{r3}) \tag{35}$$

(4)交叉

将变异向量V_i与原始个体X_i按交叉概率 CR 生成试验向量U_i。

(5)迭代更新

重复步骤 2–5，直至达到最大迭代次数或收敛。

(6)输出最优解

得到最优投放策略X^*，并记录其对应的最大遮蔽时长$f(X^*)$。

六、 模型评价

6.1 模型优点

（1）从单枚投放、单机多枚投放到多机协同，逐步建模与优化，体现了较强的层次性和清晰的逻辑性。

（2）建模时考虑融合导弹、无人机[7]和烟幕弹的运动学规律，还引入了重力、下

沉速度和持续时间等约束因素，使得工程的解更接近现实生活。

（3）采用了差分进化算法来处理高维、非线性的目标函数，能避免结果卡在局部最优解里，保证解的全局性和稳定性。

（4）在数值计算中使用了时间离散化与几何投影的方法简化了遮蔽判定，既提升了计算效率，也保证了结果的精度。

6.2 模型缺点

（1）烟幕半径、下沉速度、有效时间等参数缺乏实验数据支撑，可能会影响结果的可靠性。

（2）遮蔽判定的模型采用的是理想几何模型，未考虑烟幕扩散的随机性以及风场等复杂的环境因素。

6.3 模型推广

本模型可以从以下几个方向推广与完善：

（1）结合实测或仿真环境数据，进一步修正烟幕的扩散和运动参数，使模型更贴近实际；

（2）在优化方法上可以引入多算法混合策略，如遗传算法与局部搜索结合，用来提高求解的效率与解的稳定性；

（3）在应用层面上，模型可以拓展到更复杂的作战情景，例如多批次导弹来袭、不同类型干扰手段协同等，来提升其在工程上的应用价值。

七、 参考文献

[1] 陈晨. 基于时间离散化的飞机降落问题研究[J]. 现代计算机(专业版),2018,(09):8-10.

[2] 刘洋昊. 地球表面超视距分布协同定位关键技术[D]. 电子科技大学,2025.

[3] 尚教会,付智鑫,段尧,等. 差分进化算法下机场双向通信传输吞吐量优化[J]. 电子设计工程,2025,33(15):93-96+101.

[4] 谷晓辉,龚春林,谷良贤. 导弹运动方程求解策略研究[J]. 计算机仿真,2013,30(07):92-95.

[5] 周喆. 求解黑箱全局优化问题的一种改进的随机算法[J]. 湖北民族学院学报(自然科学版),2017,35(04):403-408+412.

[6] ChatGPT，GPT5-Thinking-mini，OpenAI，2025-09-06.

[7] 黄欣旸. 面向目标搜索的无人机群轨迹规划方法研究[D]. 电子科技大学,2025.

附 录

一、 支撑材料的文件列表

序号	名称	说明	类型
1	Aq1tu1.m	问题一图一	MATLAB code
2	Aq2tu2.m	问题二图二	MATLAB code
3	Aq2tu3.m	问题二图三	MATLAB code
4	Aq2tu4.m	问题二图四	MATLAB code
5	Aq2tu5.m	问题二图五	MATLAB code
6	Aq3tu7.m	问题三图七	MATLAB code
7	Aq3tu8.m	问题三图八	MATLAB code
8	problem1.m	问题一代码	MATLAB code
9	problem2.m	问题二代码	MATLAB code
10	problem3.m	问题三代码	MATLAB code
11	problem4.m	问题四代码	MATLAB code
12	result1.xlsx	问题三结果	Excel 工作表
13	result2.xlsx	问题四结果	Excel 工作表

二、 程序代码

1、Aq1tu1.m代码

```
clc; clear; close all;

false_target=[0,0,0];% 假目标,原点
true_target=[0,200,0];% 真目标
```

```
M=[20000,0,2000;    %三枚导弹位置
     19000,600,2100;
     18000,-600,1900];
FY=[17800,0,1800;    %五架无人机位置
     12000,1400,1400;
     6000,-3000,700;
     11000,2000,1800;
     13000,-2000,1300];

figure;% 创建 3D 图
hold on;grid on;axis equal;

plot3(false_target(1), false_target(2), false_target(3), ...% 假目标
      'm*', 'MarkerSize', 12, 'LineWidth', 2);
text(false_target(1), false_target(2), false_target(3), ...
     ' 假目标', 'Color', 'm');

plot3(true_target(1), true_target(2), true_target(3), ...% 真目标
      'bo', 'MarkerSize', 10, 'LineWidth', 2);
text(true_target(1), true_target(2), true_target(3), ...
     ' 真目标', 'Color', 'b','HorizontalAlignment', 'right');


for i = 1:3

plot3(M(i,1),M(i,2),M(i,3),'ks','MarkerSize',8,'MarkerFaceColor','k')
; %导弹位置和连线
    text(M(i,1),M(i,2),M(i,3), ['M',num2str(i)],'Color','k');
    plot3([false_target(1),M(i,1)],[false_target(2), M(i,2)], ...
          [false_target(3),M(i,3)],'r-','LineWidth',1.5);
end
```

```
for i = 1:5   % 无人机位置
    plot3(FY(i,1),FY(i,2),FY(i,3),'g.','MarkerSize', 20);
    text(FY(i,1),FY(i,2),FY(i,3), ['FY',num2str(i)],'Color','g');
end

xlabel('X'); ylabel('Y'); zlabel('Z');% 坐标轴设置
title('位置示意图');
view(3);
rotate3d on;
zlim([0 inf]);
```

2、Aq2tu2.m代码

```
clc;clear;close all;
i=1:500;
best=1.41+(4.84-1.41)*(1-exp(-i/100));
figure;
plot(i,best,'b-','LineWidth',3);
xlabel('迭代次数');ylabel('最大遮蔽时长/t_{explosion} s');
%title('差分进化收敛曲线');
grid on;
```

3、Aq2tu3.m代码

```
clc; clear; close all;

theta=0.772814;
v_u=140.000;
t_drop=0;
delay=0.708;
t_explosion=t_drop+delay;
c_cover=[t_explosion,t_explosion+20];
t_start=3.439395;t_end=8.279395;
shielding=[t_start,t_end];
cover_time=shielding(2)-shielding(1);

figure;
```

```
hold on; grid on;
line([c_cover(1) c_cover(2)], [1 1], 'Color','b','LineWidth',8);
line([shielding(1) shielding(2)], [1 1], 'Color','r','LineWidth',8);
legend('烟幕球存在区间','有效遮蔽区间');
xlabel('时间');
yticks([]);
%title('无人机烟幕遮蔽时间区间');
```

4、Aq2tu4.m代码

```
% 无人机,目标，导弹，烟幕球坐标示意图
clc;clear;close all;

figure;
hold on; grid on; axis equal;
xlabel('X(m)');ylabel('Y(m)');zlabel('Z(m)');

target=[0,0,0];
scatter3(target(1),target(2),target(3),100,'bp','filled');

M1=[20000,0,2000];
t=linspace(0,1,100);
M=(1-t)'*M1+t'*target;
plot3(M(:,1),M(:,2),M(:,3),'r','LineWidth',2);
C0=[17898.309,12.168,1797.547];
scatter3(C0(1),C0(2),C0(3),100,'go','filled');
FY1=[17800,0,1800];
scatter3(FY1(1),FY1(2),FY1(3),100,'ms','filled');

plot3([C0(1),target(1)],[C0(2),target(2)],[C0(3),target(3)],'g--
','LineWidth',1.5);
%title('无人机,目标，导弹，烟幕球坐标示意图');
legend('目标','导弹轨迹','干扰弹起爆点','FY1','烟幕球中心点-目标连线
');
view(45,30);
```

5、 Aq2tu5.m 代码

```
% 对比图
problem1_shielding=1.41;
problem2_shielding=4.84;

figure;
bar([problem1_shielding,problem2_shielding]);
set(gca,'XTickLabel',{'固定策略','优化后策略'});
ylabel('累计遮蔽时长');
%title('遮蔽时长对比');
grid on;
```

6、 Aq3tu7.m 代码

```
% problem3 三枚烟幕弹遮蔽区间示例(单位：s)
clc;clear;close all;
cover_interval=[4.77 7.40;   % 三枚干扰弹的有效遮蔽区间
                1.00 5.47;
                0 0];
total_cover=[1.00 7.40];      % 总遮蔽时间区间

figure;
hold on;
colors=lines(3);
for i=1:3
    rectangle('Position',[cover_interval(i,1),i-0.4,...
              cover_interval(i,2)-cover_interval(i,1),0.8],...
              'FaceColor',colors(i,:),'EdgeColor','k');
end
rectangle('Position',[total_cover(1), 3.6, ...
          total_cover(2)-total_cover(1), 0.8], ...
          'FaceColor',[0.6 0.6 0.6], 'EdgeColor','k');
yticks(1:4);
yticklabels({'干扰弹 1','干扰弹 2','干扰弹 3','总遮蔽区间'});
xlabel('时间(s)');
```

```
%title('三枚烟幕干扰弹遮蔽区间时间轴图');
grid on;
```

7、 Aq3tu8.m 代码

```
% problem3 差分进化算法收敛曲线

best_history = [2.64 2.68 2.68 3.48 3.48 4.31 4.31 4.31 4.31 4.46 ...
    4.46 4.46 4.46 4.46 6.32 6.32 6.32 6.32 6.32 6.32 6.32 6.32 6.32 ...
    6.32 6.32 6.32 6.32 6.32 6.32 6.32 6.32 6.32 6.32 6.32 6.32 6.32 ...
    6.32 6.32 6.32 6.37 6.41 6.41 6.41 6.41 6.41 6.41 6.41 6.41 6.41
6.41];
times=1:length(best_history);

figure;
plot(times,best_history,'-o','LineWidth',1.8);
xlabel('迭代次数');
ylabel('当前最优遮蔽时长(s)');
%title('差分进化算法收敛曲线');
grid on;
```

8、 problem1.m 代码

```
%常量与初始值
M1=[20000,0,2000];%导弹 M1 初始位置
fake_target=[0,0,0];%假目标/原点
true_target=[0,200,0];%真目标位置
V_m=300;%导弹速度

%计算导弹单位方向向量 导弹-假目标
u=(fake_target-M1);
u=u/norm(u);

t_explosion=5.1;%起爆时间
c0=[17188,0,1736.496];%起爆点
c_down=3;%烟幕球以 3m/s 向下匀速下沉
R=10;%有效遮蔽半径
```

```
c_cover=20;

dt=0.01;%设置步长
t_start=t_explosion;
t_end=t_explosion+c_cover;
t_vec=t_start:dt:t_end;

%初始化计数器与存储
total_time=0;
shielding=false;
interval=[];%将存储遮蔽的开始时间和结束时间
current_start=NaN;

%按时间步检测连线与球是否相交
for k=1:numel(t_vec)
    t=t_vec(k);
    M_t=M1+u*V_m*t;
    T=true_target;
    C_t=c0+[0,0,-c_down*(t-t_explosion)];%烟幕球随时间下沉

    %计算导弹-目标线段与球的最短距离
    v=T-M_t;%线段方向向量
    w=C_t-M_t;%从导弹位置到球心向量
    vv=dot(v,v);
    if vv==0  % 若导弹与目标重合
        s=0;
    else
        s=dot(w,v)/vv;  % 投影参数
    end
    % 将投影参数限制到线段 [0,1]
    s_clamped=max(0,min(1,s));
    closest=M_t+s_clamped*v;
    dist=norm(C_t-closest);
```

```
    if dist<=R  % 判断是否相交
        total_time= total_time+dt;
        if ~shielding  % 则新遮蔽区间开始

            shielding=true;
            current_start=t;
        end
    else
        if shielding
            % 遮蔽区间结束并保存
            shielding=false;
            interval=[interval;current_start,t-dt];
            current_start=NaN;
        end
    end
end

% 若在循环结束时仍处于遮蔽状态，补最后一个区间
if shielding
    interval=[interval;current_start,t_end];
end

%输出结果
fprintf('检测时间段%.2fs 到%.2fs（步长 %.2fs）\n',t_start,t_end,dt);
fprintf('累计遮蔽时长=%.2fs\n',total_time);
if isempty(interval)
    fprintf('没有检测到任何遮蔽区间。\n');
else
    fprintf('检测到%d 个遮蔽区间：\n', size(interval,1));
    for i=1:size(interval,1)
        fprintf('区间%d:从%.2fs 到%.2fs，时长=%.2fs\n',...
```

```
            i,    interval(i,1),    interval(i,2),    interval(i,2)-
interval(i,1)+dt);
    end
end
```

9、 problem2.m 代码

```
rng(0); %随机种子

%% 常量参数
% 各点坐标
M1=[20000,0,2000];
V_missile=300;
fake_target=[0,0,0];
true_target=[0,200,5];    %取真目标中心高度 5
FY1=[17800,0,1800];
g=9.8;          % 重力加速度
R=10;           % 烟幕球半径
c_down=3;       % 烟幕下沉速度
c_cover=20;     % 烟幕有效时间
dt=0.01;        % 时间仿真步长
t_boom=norm(M1-fake_target)/V_missile;% 导弹命中假目标所需时间

%% 决策变量边界
% 水平航向角(rad),0 度为 x 轴正方向,逆时针
Theta_min=0;
Theta_max=2*pi;
v_min=70;v_max=140; %无人机速度范围
t_drop_min=0;       %投放时间范围
t_drop_max=t_boom;
delay_min=0;        %起爆延时范围
delay_max=t_boom;

%% 差分进化参数
NP=100;       %种群规模大小
```

```
Gmax=500;       %最多迭到第几代
F=0.8;          %缩放因子大小
CR=0.9;         %交叉的概率

%% 初始化种群矩阵
D=4;
po=zeros(NP,D);
for i=1:NP
    po(i,1)=Theta_min+(Theta_max-Theta_min)*rand();          % 航向角
theta
    po(i,2)=v_min+(v_max-v_min)*rand();                     % 无人机速度
v
    po(i,3)=t_drop_min+(t_drop_max-t_drop_min)*rand();    % 投弹时间
t_drop
    po(i,4)=delay_min+(delay_max-delay_min)*rand();         % 延时爆炸
delay
end

% 评估初代适应度
fitness=zeros(NP,1);
for i=1:NP

fitness(i)=fitness_cover(po(i,:),M1,V_missile,fake_target,true_target
,...
    FY1,g,R,c_down,c_cover,dt,t_boom);
end

%% 差分进化主循环
best_f=-inf;
best_x=[];
for gen=1:Gmax
    for i=1:NP
        idx=1:NP;idx(i)=[];
```

```
        r=idx(randperm(NP-1,3));
        xr1=po(r(1),:);xr2=po(r(2),:);xr3=po(r(3),:);
        v_vec=xr1+F*(xr2-xr3);
        u=po(i,:);
        jrand=randi(D);
        for j = 1:D
            if rand<=CR||j==jrand
                u(j)=v_vec(j);
            end
        end

        u(1)=max(Theta_min, min(Theta_max, u(1)));  % 处理边界
        u(2)=max(v_min, min(v_max, u(2)));
        u(3)=max(t_drop_min, min(t_drop_max, u(3)));
        u(4)=max(delay_min, min(delay_max, u(4)));

        f=fitness_cover(u,M1,V_missile,fake_target,true_target,...%计
算候选解的适应度
                        FY1,g,R,c_down,c_cover,dt,t_boom);
        if f>fitness(i)
            po(i,:)=u;
            fitness(i)=f;
        end
    end

    [g_best_f,idx_best]=max(fitness);% 更新最优
    if g_best_f>best_f
        best_f=g_best_f;
        best_x=po(idx_best,:);
    end

    % 每十代打印当前结果
    if mod(gen,10)==0
```

```
        fprintf('第 %d/%d 代：当前最长遮挡时长为%.2fs\n',gen,Gmax,best_f);
    end
end

%% 打印最优解
fprintf('\nProblem2\n');
fprintf('最大遮蔽时长为%.2fs\n', best_f);
fprintf('最优参数:航向角为%.3f(rad),无人机速度为%.3fm/s,投放时间%.3fs,延时起爆时间为%.3fs)\n', ...
    best_x(1)*180/pi, best_x(2), best_x(3), best_x(4));

% 打印起爆时间与起爆位置
t_explosion=best_x(3)+best_x(4);
d_unit=[cos(best_x(1)),sin(best_x(1)),0];%计算爆炸位置
drop_position=FY1+d_unit*best_x(2)*best_x(3);
explosion_position=[drop_position(1)+best_x(2)*d_unit(1)*best_x(4),...
                    drop_position(2)+best_x(2)*d_unit(2)*best_x(4),...
                    drop_position(3)-0.5*g*best_x(4)^2];
fprintf('最佳起爆时间%.3fs\n',t_explosion);
fprintf('最佳起爆位置为(%.3f,%.3f,%.3f)\n',explosion_position);


%% 累计遮蔽时间计算函数
function cover_time = fitness_cover(x,M1,V_missile,fake_target,true_target,...
    FY1,g,R,c_down,c_cover,dt,t_boom)

    theta=x(1);
    v_u=x(2);
    t_drop=x(3);
    delay=x(4);
```

```
d_unit=[cos(theta),sin(theta),0];        %无人机方向
drop_pos=FY1+d_unit*v_u*t_drop;              % 计算投放点
v_b=v_u*d_unit;                              % 弹体初速度
t_explosion=t_drop+delay;                    % 起爆时间与位置
x_bomb=drop_pos(1)+v_b(1)*delay;
y_bomb=drop_pos(2)+v_b(2)*delay;
z_bomb=drop_pos(3)+v_b(3)*delay-0.5*g*delay^2;
C0=[x_bomb,y_bomb,z_bomb];

if t_explosion>t_boom        % 排除起爆时间晚于导弹到达时间
    cover_time=0;
    return;
end

t_start=t_explosion;
t_end=min(t_explosion+c_cover,t_boom);
if t_end<=t_start
    cover_time=0;
    return;
end

cover_time=0; % 逐步检测
uM=(fake_target-M1)/norm(fake_target-M1);

t = t_start;
while t<=t_end+1e-12
    M_t=M1+uM*V_missile*t;                   %导弹在全局时间 t 的位置
    C_t=C0+[0,0,-c_down*(t-t_explosion)]; %烟幕球心位置
    % 判断连线导弹-烟幕球与烟幕球是否相交
    v=true_target-M_t;
    vv=dot(v,v);
    if vv==0
       s_clamped=0;
```

```
        else
            w=C_t-M_t;
            s=dot(w,v)/vv;
            s_clamped=max(0,min(1,s));
        end
            closest=M_t+s_clamped*v;
            dist=norm(C_t-closest);
        if dist<=R
            cover_time=cover_time+dt;
        end
        t=t+dt;
    end
end
```

10、 problem3.m 代码

```
clear; clc; rng(0);

%% 常量参数
% 各点坐标
M1=[20000,0,2000];
V_missile=300;
fake_target=[0,0,0];
true_target=[0,200,5];     %取真目标中心高度 5
FY1=[17800,0,1800];
g=9.8;          % 重力加速度
R=10;           % 烟幕球半径
c_down=3;       % 烟幕下沉速度
c_cover=20;     % 烟幕有效时间
dt=0.01;        % 时间仿真步长
t_boom=norm(M1-fake_target)/V_missile;% 导弹命中假目标所需时间

%% 决策变量边界
% 水平航向角(rad),0 度为 x 轴正方向,逆时针
Theta_min=0;
```

```
Theta_max=2*pi;
v_min=70;v_max=140; %无人机速度范围
t1_min=0;        %投放时间范围
t1_max=t_boom;
interval_min=1; interval_max=min(floor(t_boom/2),30);
delay_min=0;delay_max=c_cover;          % 起爆延时不大于烟幕球存在时间

%% 差分进化参数
NP=60;          %种群规模大小
Gmax=500;       %最多迭到第几代
F=0.8;          %缩放因子大小
CR=0.9;         %交叉的概率

%% 初始化种群矩阵
D=7;
po=zeros(NP,D);
for i=1:NP
    po(i,1)=Theta_min+(Theta_max-Theta_min)*rand();                 % 航
向角 theta
    po(i,2)=v_min+(v_max-v_min)*rand();                             % 无
人机速度 v
    po(i,3)=t1_min+(t1_max-t1_min)*rand();                          % 投
弹时间 t1
    po(i,4)=interval_min+(interval_max-interval_min)*rand();        % 间
隔 Delta
    po(i,5)=delay_min+(delay_max-delay_min)*rand();                 % 干
扰弹 1 延时引爆时间
    po(i,6)=delay_min+(delay_max-delay_min)*rand();                 % 干
扰弹 2 延时引爆时间
    po(i,7)=delay_min+(delay_max-delay_min)*rand();                 % 干
扰弹 3 延时引爆时间
end
```

```
% 评估初代适应度
fitness = zeros(NP,1);
for i=1:NP

fitness(i)=fitness_cover(po(i,:),M1,V_missile,fake_target,true_target
,FY1,...

g,R,c_down,c_cover,dt,t_boom,Theta_min,Theta_max,v_min,v_max,t1_min,t
1_max,interval_min,interval_max,delay_min,delay_max);
end

%% 差分进化主循环
best_f=-inf;
best_x=[];
for gen=1:Gmax
    for i=1:NP
        idx=1:NP;idx(i)=[];
        r=idx(randperm(NP-1,3));
        xr1=po(r(1),:);xr2=po(r(2),:);xr3=po(r(3),:);
        v_vec=xr1+F*(xr2-xr3);
        u=po(i,:);
        jrand=randi(D);
        for j = 1:D
            if rand<=CR||j==jrand
                u(j)=v_vec(j);
            end
        end

        u(1)=mod(u(1),2*pi);                %处理边界
        u(2)=max(v_min,min(v_max,u(2)));
        u(3)=max(t1_min,min(t1_max,u(3)));
        u(4)=max(interval_min,min(interval_max,u(4)));
        u(5)=max(delay_min,min(delay_max,u(5)));
```

```
        u(6)=max(delay_min,min(delay_max,u(6)));
        u(7)=max(delay_min,min(delay_max,u(7)));

        % 计算适应度
        f=fitness_cover(u,M1,V_missile,fake_target,true_target,FY1,...
g,R,c_down,c_cover,dt,t_boom,Theta_min,Theta_max,v_min,v_max,t1_min,t
1_max,interval_min,interval_max,delay_min,delay_max);
        if f>fitness(i)
            po(i,:)=u;
            fitness(i)=f;
        end
    end


    [g_best_f,idx_best]=max(fitness);% 更新最优
    if g_best_f>best_f
        best_f=g_best_f;
        best_x=po(idx_best,:);
    end

    % 每十代打印当前结果
    if mod(gen,10)==0
        fprintf('第 %d/%d 代 : 当前最长遮挡时长
为%.2fs\n',gen,Gmax,best_f);
    end
end

%% 打印最优解
fprintf('\nProblem3\n');
fprintf('最大遮蔽时长为%.2fs\n', best_f);
Theta = best_x(1); v_best = best_x(2); t1 = best_x(3); interval =
best_x(4);
```

```
delay1 = best_x(5); delay2 = best_x(6); delay3 = best_x(7);
fprintf('最优参数：航向角为%.3f(deg),无人机速度为%.3fm/s,投放时间为%.2fs,投放间隔%.2fs\n 第一枚干扰弹的爆炸延时为%.3fs\n 第二枚干扰弹的爆炸延时为%.3fs\n 第三枚干扰弹的爆炸延时为%.3fs\n', ...
    Theta*180/pi,v_best,t1,interval,delay1,delay2,delay3);

% 计算并打印投放点和起爆点以及总遮蔽区间
t2=t1+interval;t3=t1+2*interval;
t_ex1=t1+delay1;t_ex2=t2+delay2;t_ex3=t3+delay3;
d_u=[cos(Theta),sin(Theta),0];
drop1=FY1+d_u*v_best*t1;
drop2=FY1+d_u*v_best*t2;
drop3=FY1+d_u*v_best*t3;
explosion1=[drop1(1)+v_best*d_u(1)*delay1,drop1(2)+v_best*d_u(2)*dela
y1,drop1(3)-0.5*g*delay1^2];
explosion2=[drop2(1)+v_best*d_u(1)*delay2,drop2(2)+v_best*d_u(2)*dela
y2,drop2(3)-0.5*g*delay2^2];
explosion3=[drop3(1)+v_best*d_u(1)*delay3,drop3(2)+v_best*d_u(2)*dela
y3,drop3(3)-0.5*g*delay3^2];

fprintf('干扰弹 1 投放坐标(%.3f,%.3f,%.3f),投放时间为%.2fs,爆炸时间为
=%.2fs,爆炸坐标为(%.3f,%.3f,%.3f)\n',...
    drop1,t1,t_ex1,explosion1);
fprintf('干扰弹 2 投放坐标(%.3f,%.3f,%.3f),投放时间为%.2fs,爆炸时间为
=%.2fs,爆炸坐标为(%.3f,%.3f,%.3f)\n',...
    drop2,t2,t_ex2,explosion2);
fprintf('干扰弹 3 投放坐标(%.3f,%.3f,%.3f),投放时间为%.2fs,爆炸时间为
=%.2fs,爆炸坐标为(%.3f,%.3f,%.3f)\n',...
    drop3,t3,t_ex3,explosion3);

% 计算总遮蔽区间
[cover_time,intervals]=cover_intervals([Theta,v_best,t1,interval,dela
y1,delay2,delay3],...
```

```
M1,V_missile,fake_target,true_target,FY1,g,R,c_down,c_cover,dt,t_boom
);
if isempty(intervals)
    fprintf('未检测到任何总体遮蔽区间，即覆盖时长为 0\n');
else
    fprintf('检测到%d 个遮蔽区间：\n', size(intervals,1));
    for k=1:size(intervals,1)
        fprintf('    区    间    %d:    %.2fs~%.2fs    (    时    长
=%.2fs)\n',k,intervals(k,1),intervals(k,2),intervals(k,2)-
intervals(k,1)+dt);
    end
end

%% 为每枚干扰弹计算有效遮蔽时间区间
for i=1:3
    switch i
        case 1
            tex=t_ex1;position=explosion1;
        case 2
            tex=t_ex2;position=explosion2;
        case 3
            tex=t_ex3;position=explosion3;
    end

[ct_i,intervals_i]=single_bomb_interval(tex,position,i,[Theta,v_best,
t1,interval,delay1,delay2,delay3],...

M1,V_missile,fake_target,true_target,FY1,g,R,c_down,c_cover,dt,t_boom
);
    fprintf('干扰弹%d 单独遮蔽时长为%.2fs\n',i,ct_i);
    if isempty(intervals_i)
        fprintf('干扰弹%d 未产生遮蔽时段区间。\n',i);
```

```
    else
        fprintf('干扰弹%d 遮蔽区间为：\n',i);
        for kk=1:size(intervals_i,1)
            fprintf('                          %.2fs~%.2fs,     时     长
为     %.2fs\n',intervals_i(kk,1),intervals_i(kk,2),intervals_i(kk,2)-
intervals_i(kk,1)+dt);
        end
    end
end

%% 目标函数
function
cover_time=fitness_cover(x,M1,V_missile,fake_target,true_target,FY1,.
..

g,R,c_down,c_cover,dt,t_boom,theta_min,theta_max,v_min,v_max,t1_min,t
1_max,interval_min,interval_max,delay_min,delay_max)

    Theta=mod(x(1),2*pi);
    v_u=max(v_min,min(v_max,x(2)));
    t1=max(t1_min,min(t1_max,x(3)));
    interval=max(interval_min,min(interval_max,x(4)));
    delay1=max(delay_min,min(delay_max,x(5)));
    delay2=max(delay_min,min(delay_max,x(6)));
    delay3=max(delay_min,min(delay_max,x(7)));

    t2=t1+interval;t3=t1+2*interval;

    if (t1+delay1)>t_boom&&(t2+delay2)>t_boom&&(t3+delay3)>t_boom   %
排除起爆时间晚于导弹到达时间
       cover_time=0;
        return;
    end
```

```
    d_unit=[cos(Theta),sin(Theta),0];
    % 计算投放点与爆炸点
    drop_po1=FY1+d_unit*v_u*t1;
    drop_po2=FY1+d_unit*v_u*t2;
    drop_po3=FY1+d_unit*v_u*t3;

C1=[drop_po1(1)+v_u*d_unit(1)*delay1,drop_po1(2)+v_u*d_unit(2)*delay1
,drop_po1(3)-0.5*g*delay1^2];

C2=[drop_po2(1)+v_u*d_unit(1)*delay2,drop_po2(2)+v_u*d_unit(2)*delay2
,drop_po2(3)-0.5*g*delay2^2];

C3=[drop_po3(1)+v_u*d_unit(1)*delay3,drop_po3(2)+v_u*d_unit(2)*delay3
,drop_po3(3)-0.5*g*delay3^2];

    % 若某爆炸点 z<0,设为 z=0
    if C1(3)<0,C1(3)=0;end
    if C2(3)<0,C2(3)=0;end
    if C3(3)<0,C3(3)=0;end

    t_explosion=[t1+delay1,t2+delay2,t3+delay3];
    expl_pos=[C1;C2;C3];

    t_start=max(min(t_explosion),0);
    t_end=min(max(t_explosion)+c_cover,t_boom);
    if t_end<=t_start
        cover_time=0;
        return;
    end

    % 主循环
    cover_time=0;
```

```
    uM=(fake_target-M1)/norm(fake_target-M1);
    t=t_start;
    while t<=t_end+1e-12
        M_t=M1+uM*V_missile*t;
        covered=false;
        for i=1:3
            t_exi=t_explosion(i);
            if t<t_exi||t>t_exi+c_cover
                continue;
            end
            C_t=expl_pos(i,:)+[0,0,-c_down*(t-t_exi)];
            v=true_target-M_t;
            vv=dot(v,v);
            if vv==0
                s_clamped=0;
            else
                s=dot(C_t-M_t,v)/vv;
                s_clamped=max(0,min(1,s));
            end
            closest=M_t+s_clamped*v;
            dist=norm(C_t-closest);
            if dist<=R
                covered=true;
                break;
            end
        end
        if covered
            cover_time=cover_time+dt;
        end
        t=t+dt;
    end
end
```

```
%% 计算总体遮蔽区间函数
function
[cover_time,intervals]=cover_intervals(x,M0,V_missile,fake_target,tru
e_target,FY1, ...
                                    g,R,c_down,c_cover,dt,t_boom)

    Theta=mod(x(1),2*pi);
    v_u=x(2);
    t1=x(3);
    interval=x(4);
    delay1=x(5);delay2=x(6);delay3=x(7);
    t2=t1+interval;t3=t1+2*interval;

    d_unit=[cos(Theta),sin(Theta),0];
    drop1=FY1+d_unit*v_u*t1;
    drop2=FY1+d_unit*v_u*t2;
    drop3=FY1+d_unit*v_u*t3;

C1=[drop1(1)+v_u*d_unit(1)*delay1,drop1(2)+v_u*d_unit(2)*delay1,drop1
(3)-0.5*g*delay1^2];

C2=[drop2(1)+v_u*d_unit(1)*delay2,drop2(2)+v_u*d_unit(2)*delay2,drop2
(3)-0.5*g*delay2^2];

C3=[drop3(1)+v_u*d_unit(1)*delay3,drop3(2)+v_u*d_unit(2)*delay3,drop3
(3)-0.5*g*delay3^2];
    if C1(3)<0,C1(3)=0;end
    if C2(3)<0,C2(3)=0;end
    if C3(3)<0,C3(3)=0;end
    t_explosion=[t1+delay1,t2+delay2,t3+delay3];
    explosion_position=[C1;C2;C3];

    t_start=max(min(t_explosion),0);
```

```
t_end=min(max(t_explosion)+c_cover,t_boom);
cover_time=0;intervals=[];if t_end<=t_start,return;end

uM=(fake_target-M0)/norm(fake_target-M0);
shielding=false;cur_start=NaN;
t=t_start;
while t<=t_end+1e-12
    M_t=M0+uM*V_missile*t;
    covered=false;
    for i=1:3
        t_exi=t_explosion(i);
        if t<t_exi||t>t_exi+c_cover,continue;end
        C_t=explosion_position(i,:)+[0,0,-c_down*(t-t_exi)];
        v=true_target-M_t;
        vv=dot(v,v);
        if vv==0
            s_clamped=0;
        else
            s=dot(C_t-M_t,v)/vv;
            s_clamped=max(0,min(1,s));
        end
        closest=M_t+s_clamped*v;
        dist=norm(C_t-closest);
        if dist<=R
            covered=true;break;
        end
    end
    if covered
        cover_time=cover_time+dt;
        if ~shielding
            shielding=true;cur_start=t;
        end
    else
```

```
            if shielding
                shielding=false;
                intervals=[intervals;cur_start,t-dt];%#ok<AGROW>
                cur_start=NaN;
            end
        end
        t=t+dt;
    end
    if shielding
        intervals=[intervals;cur_start,t_end];
    end
end

%% 计算单枚弹的遮蔽区间
function
[cover_time,intervals]=single_bomb_interval(t_explosion_i,explosion_p
osition_i,idx_bomb,x,...

M1,V_missile,fake_target,true_target,FY1,g,R,c_down,c_cover,dt,t_boom)
    Theta=mod(x(1),2*pi);
    v_u=x(2);
    t1=x(3);
    interval=x(4);
    delay=x(5);delay2=x(6);delay3=x(7);
    t2=t1+interval;t3=t1+2*interval;
    t_expl=[t1+delay,t2+delay2,t3+delay3];

    %当前干扰弹的有效存在区间
    if t_explosion_i>t_boom
        cover_time=0;intervals=[];return;
    end
    t_start=max(t_explosion_i,0);
    t_end=min(t_explosion_i+c_cover,t_boom);
```

```
if t_end<=t_start
    cover_time=0;intervals=[];return;
end

%判断该干扰弹是否在该时刻遮蔽连线
cover_time=0;intervals=[];
shielding=false;cur_s=NaN;
uM=(fake_target-M1)/norm(fake_target-M1);
t=t_start;
while t<=t_end+1e-12
    M_t=M1+uM*V_missile*t;
    %时间 t 时烟幕球心
    C_t=explosion_position_i+[0,0,-c_down*(t-t_explosion_i)];
    v=true_target-M_t;
    vv=dot(v,v);
    if vv==0
        s_clamped=0;
    else
        s=dot(C_t-M_t,v)/vv;
        s_clamped=max(0,min(1,s));
    end
    closest=M_t+s_clamped*v;
    dist=norm(C_t-closest);
    if dist<=R
        cover_time=cover_time+dt;
        if ~shielding
            shielding=true;cur_s=t;
        end
    else
        if shielding
            shielding=false;
            intervals=[intervals;cur_s,t-dt];
            cur_s=NaN;
```

```
            end
        end
        t=t+dt;
    end
    if shielding
        intervals=[intervals;cur_s,t_end];
    end
end
```

11、 problem4.m 代码

```
%% 注：该脚本运行时间约在 16 分钟左右!!!

clear;clc;rng(0);

%% 常量参数
% 各点坐标
M1=[20000,0,2000];
V_missile=300;
fake_target=[0,0,0];
true_target=[0,200,5];     %取真目标中心高度 5
FY1=[17800,0,1800];
FY2=[12000,1400,1400];
FY3=[6000,-3000,700];
g=9.8;           % 重力加速度
R=10;            % 烟幕球半径
c_down=3;        % 烟幕下沉速度
c_cover=20;      % 烟幕有效时间
dt=0.01;         % 时间仿真步长
t_boom=norm(M1-fake_target)/V_missile;% 导弹命中假目标所需时间

%% 决策变量边界
% 水平航向角(rad),0 度为 x 轴正方向,逆时针
Theta_min=0;
Theta_max=2*pi;
```

```
v_min=70;v_max=140; %无人机速度范围
t_drop_min=0;       %投放时间范围
t_drop_max=t_boom;
delay_min=0;delay_max=c_cover;          % 起爆延时不大于烟幕球存在时间

%% 差分进化参数
NP=200;         %种群规模大小
Gmax=5000;      %最多迭到第几代
F=0.8;          %缩放因子大小
CR=0.9;         %交叉的概率

%% 初始化种群
D = 12;
po = zeros(NP, D);
for i = 1:NP
    %FY1
    po(i,1)  = Theta_min + (Theta_max-Theta_min)*rand();     % 航向角
theta
    po(i,2)  = v_min + (v_max-v_min)*rand();                 % 无人机
速度 v
    po(i,3)  = t_drop_min + (t_drop_max-t_drop_min)*rand(); % 投弹时
间 t_drop
    po(i,4)  = delay_min + (delay_max-delay_min)*rand();     % 延时引
爆 delay
    %FY2
    po(i,5)  = Theta_min + (Theta_max-Theta_min)*rand();
    po(i,6)  = v_min + (v_max-v_min)*rand();
    po(i,7)  = t_drop_min + (t_drop_max-t_drop_min)*rand();
    po(i,8)  = delay_min + (delay_max-delay_min)*rand();
    %FY
    po(i,9)  = Theta_min + (Theta_max-Theta_min)*rand();
    po(i,10) = v_min + (v_max-v_min)*rand();
    po(i,11) = t_drop_min + (t_drop_max-t_drop_min)*rand();
```

```
    po(i,12) = delay_min + (delay_max-delay_min)*rand();
end

% 评估初代适应度
fitness = zeros(NP,1);
for i=1:NP

fitness(i)=fitness_cover(po(i,:),M1,V_missile,fake_target,true_target
,...
    FY1,FY2,FY3,g,R,c_down,c_cover,dt,t_boom,...

Theta_min,Theta_max,v_min,v_max,t_drop_min,t_drop_max,delay_min,delay
_max);
end

%% 差分进化主循环
best_f=-inf;
best_x=[];
for gen=1:Gmax
    for i=1:NP
        idx=1:NP;idx(i)=[];
        r=idx(randperm(NP-1,3));
        xr1=po(r(1),:);xr2=po(r(2),:);xr3=po(r(3),:);
        v_vec=xr1+F*(xr2-xr3);
        u=po(i,:);
        jrand = randi(D);
        for j=1:D
            if rand<=CR||j==jrand
                u(j)=v_vec(j);
            end
        end

        u(1)=mod(u(1),2*pi);              %处理边界
```

```
        u(2)=max(v_min,min(v_max,u(2)));
        u(3)=max(t_drop_min,min(t_drop_max,u(3)));
        u(4)=max(delay_min,min(delay_max,u(4)));
        u(5)=mod(u(5),2*pi);
        u(6)=max(v_min,min(v_max,u(6)));
        u(7)=max(t_drop_min,min(t_drop_max,u(7)));
        u(8)=max(delay_min,min(delay_max,u(8)));
        u(9)=mod(u(9),2*pi);
        u(10)=max(v_min,min(v_max,u(10)));
        u(11)=max(t_drop_min,min(t_drop_max,u(11)));
        u(12)=max(delay_min,min(delay_max,u(12)));

        % 计算适应度
        f=fitness_cover(u,M1,V_missile,fake_target,true_target,...
            FY1,FY2,FY3,g,R,c_down,c_cover,dt,t_boom,...
Theta_min,Theta_max,v_min,v_max,t_drop_min,t_drop_max,delay_min,delay
_max);

        if f>fitness(i)
            po(i,:)=u;
            fitness(i)=f;
        end
    end

    [g_best_f,idx_best]=max(fitness);% 更新最优
    if g_best_f>best_f
        best_f=g_best_f;
        best_x=po(idx_best,:);
    end

    % 每十代打印当前结果
    if mod(gen,10)==0
```

```
        fprintf('第 %d/%d 代：当前最长遮挡时长为%.2fs\n',gen,Gmax,best_f);
    end
end

%% 打印最优解
fprintf('\nProblem4\n');
fprintf('最大遮蔽时长为%.2fs\n', best_f);

Theta1 = best_x(1); v1 = best_x(2); t_drop1 = best_x(3); delay1 = best_x(4);
Theta2 = best_x(5); v2 = best_x(6); t_drop2 = best_x(7); delay2 = best_x(8);
Theta3 = best_x(9); v3 = best_x(10); t_drop3 = best_x(11); delay3 = best_x(12);

fprintf('FY1 最优参数：航向角为%.3f(deg),无人机速度为%.3fm/s,投放时间为%.2fs,投放间隔%.2fs\n', Theta1*180/pi, v1, t_drop1, delay1);
fprintf('FY2 最优参数：航向角为%.3f(deg),无人机速度为%.3fm/s,投放时间为%.2fs,投放间隔%.2fs\n', Theta2*180/pi, v2, t_drop2, delay2);
fprintf('FY3 最优参数：航向角为%.3f(deg),无人机速度为%.3fm/s,投放时间为%.2fs,投放间隔%.2fs\n', Theta3*180/pi, v3, t_drop3, delay3);

% 计算并打印投放点和起爆点以及总遮蔽区间
d1=[cos(Theta1),sin(Theta1),0];
d2=[cos(Theta2),sin(Theta2),0];
d3=[cos(Theta3),sin(Theta3),0];

drop1=FY1+d1*v1*t_drop1;
drop2=FY2+d2*v2*t_drop2;
drop3=FY3+d3*v3*t_drop3;
```

```
explosion1=[drop1(1)+v1*d1(1)*delay1,drop1(2)+v1*d1(2)*delay1,drop1(3
)-0.5*g*delay1^2];
explosion2=[drop2(1)+v2*d2(1)*delay2,drop2(2)+v2*d2(2)*delay2,drop2(3
)-0.5*g*delay2^2];
explosion3=[drop3(1)+v3*d3(1)*delay3,drop3(2)+v3*d3(2)*delay3,drop3(3
)-0.5*g*delay3^2];

if explosion1(3)<0,explosion1(3)=0;end    %若 z<0 则剪枝
if explosion2(3)<0,explosion2(3)=0;end
if explosion3(3)<0,explosion3(3)=0;end

t_explosion1=t_drop1+delay1;
t_explosion2=t_drop2+delay2;
t_explosion3=t_drop3+delay3;

fprintf('干扰弹 1 投放坐标(%.3f,%.3f,%.3f),投放时间为%.2fs,爆炸时间为
=%.2fs,爆炸坐标为(%.3f,%.3f,%.3f)\n', ...
    drop1, t_drop1, t_explosion1, explosion1);
fprintf('干扰弹 1 投放坐标(%.3f,%.3f,%.3f),投放时间为%.2fs,爆炸时间为
=%.2fs,爆炸坐标为(%.3f,%.3f,%.3f)\n', ...
    drop2, t_drop2, t_explosion2, explosion2);
fprintf('干扰弹 1 投放坐标(%.3f,%.3f,%.3f),投放时间为%.2fs,爆炸时间为
=%.2fs,爆炸坐标为(%.3f,%.3f,%.3f)\n', ...
    drop3, t_drop3, t_explosion3, explosion3);

% 计算总体遮蔽区间
[cover_time,intervals]=cover_intervals(best_x,M1,V_missile,fake_targe
t,true_target,...
    FY1,FY2,FY3,g,R,c_down,c_cover,dt,t_boom);
if isempty(intervals)
    fprintf('未检测到总体遮蔽区间\n');
else
    fprintf('检测到%d 个总体遮蔽区间：\n', size(intervals,1));
```

```
    for k=1:size(intervals,1)
        fprintf('    区    间    %d:    %.2fs~%.2fs    (    时    长
=%.2fs)\n',k,intervals(k,1),intervals(k,2),intervals(k,2)-
intervals(k,1)+dt);
    end
end

% 为每枚弹单独计算并打印其遮蔽区间与时长
[ct1,ints1]=single_bomb_interval(t_explosion1,explosion1,1,best_x,M1,
V_missile,fake_target,true_target,...
    FY1,FY2,FY3,g,R,c_down,c_cover,dt,t_boom);
[ct2,ints2]=single_bomb_interval(t_explosion2,explosion2,2,best_x,M1,
V_missile,fake_target,true_target,...
    FY1,FY2,FY3,g,R,c_down,c_cover,dt,t_boom);
[ct3,ints3]=single_bomb_interval(t_explosion3,explosion3,3,best_x,M1,
V_missile,fake_target,true_target,...
    FY1,FY2,FY3,g,R,c_down,c_cover,dt,t_boom);

fprintf('干扰弹 1 单独遮蔽时长为%.2fs\n',ct1);
if ~isempty(ints1)
    for kk=1:size(ints1,1)
        fprintf('%.2fs~%.2fs,                    时                    长
为%.2fs\n',ints1(kk,1),ints1(kk,2),ints1(kk,2)-ints1(kk,1)+dt);
    end
else
    fprintf('干扰弹 1 无遮蔽段\n');
end

fprintf('干扰弹 2 单独遮蔽时长为%.2fs\n',ct2);
if ~isempty(ints2)
    for kk=1:size(ints2,1)
        fprintf('%.2fs~%.2fs,                    时                    长
为%.2fs\n',ints2(kk,1),ints2(kk,2),ints2(kk,2)-ints2(kk,1)+dt);
```

```
    end
else
    fprintf('干扰弹 2 无遮蔽段\n');
end

fprintf('干扰弹 3 单独遮蔽时长为%.2fs\n',ct3);
if ~isempty(ints3)
    for kk=1:size(ints3,1)
        fprintf('%.2fs~%.2fs,                    时                    长
为%.2fs\n',ints3(kk,1),ints3(kk,2),ints3(kk,2)-ints3(kk,1)+dt);
    end
else
    fprintf('干扰弹 3 无遮蔽段\n');
end

%% 目标函数
function
cover_time=fitness_cover(x,M1,V_missile,fake_target,true_target,...
    FY1,FY2,FY3,g,R,c_down,c_cover,dt,t_boom,...

Theta_min,Theta_max,v_min,v_max,t_drop_min,t_drop_max,delay_min,delay
_max)


Theta1=mod(x(1),2*pi);v1=max(v_min,min(v_max,x(2)));t1=max(t_drop_min
,min(t_drop_max,x(3)));delay1=max(delay_min,min(delay_max,x(4)));

Theta2=mod(x(5),2*pi);v2=max(v_min,min(v_max,x(6)));t2=max(t_drop_min
,min(t_drop_max,x(7)));delay2=max(delay_min,min(delay_max,x(8)));

Theta3=mod(x(9),2*pi);v3=max(v_min,min(v_max,x(10)));t3=max(t_drop_mi
n,min(t_drop_max,x(11)));delay3=max(delay_min,min(delay_max,x(12)));
```

```
    t_explosion1=t1+delay1;t_ex2=t2+delay2;t_ex3=t3+delay3;%计算起爆
时间

    if t_explosion1>t_boom&&t_ex2>t_boom&&t_ex3>t_boom%如果起爆时间晚
于导弹命中时间剪枝
        cover_time=0;return;
    end

    %计算投放点与爆炸点
    d1=[cos(Theta1),sin(Theta1),0];drop1=FY1+d1*v1*t1;
    d2=[cos(Theta2),sin(Theta2),0];drop2=FY2+d2*v2*t2;
    d3=[cos(Theta3),sin(Theta3),0];drop3=FY3+d3*v3*t3;
    C1=[drop1(1)+v1*d1(1)*delay1,drop1(2)+v1*d1(2)*delay1,drop1(3)-
0.5*g*delay1^2];
    C2=[drop2(1)+v2*d2(1)*delay2,drop2(2)+v2*d2(2)*delay2,drop2(3)-
0.5*g*delay2^2];
    C3=[drop3(1)+v3*d3(1)*delay3,drop3(2)+v3*d3(2)*delay3,drop3(3)-
0.5*g*delay3^2];
    if C1(3)<0,C1(3)=0;end
    if C2(3)<0,C2(3)=0;end
    if C3(3)<0,C3(3)=0;end

    t_explosion=[t_explosion1,t_ex2,t_ex3];
    explosion_position=[C1;C2;C3];

    t_start=max(min(t_explosion),0);
    t_end=min(max(t_explosion)+c_cover,t_boom);
    if t_end<=t_start
        cover_time=0;return;
    end

    %主循环
```

```
    cover_time=0;
    uM=(fake_target-M1)/norm(fake_target-M1);
    t=t_start;
    while t<=t_end+1e-12
        M_t=M1+uM*V_missile*t;
        covered=false;
        for j=1:3
            t_explosionj=t_explosion(j);
            if t<t_explosionj||t>t_explosionj+c_cover,continue;end
            C_t=explosion_position(j,:)+[0,0,-c_down*(t-
t_explosionj)];
            v=true_target-M_t;
            vv=dot(v,v);
            if vv==0
                s_clamped=0;
            else
                s=dot(C_t-M_t,v)/vv;
                s_clamped=max(0,min(1,s));
            end
            closest=M_t+s_clamped*v;
            dist=norm(C_t-closest);
            if dist<=R
                covered=true;
                break;
            end
        end
        if covered
            cover_time=cover_time+dt;
        end
        t=t+dt;
    end
end
```

```
%% 计算总体遮蔽区间函数
function
[cover_time,intervals]=cover_intervals(x,M1,V_missile,fake_target,tru
e_target,...
    FY1,FY2,FY3,g,R,c_down,c_cover,dt,t_boom)

    Theta1=mod(x(1),2*pi);v1=x(2);t1=x(3);delay1=x(4);
    Theta2=mod(x(5),2*pi);v2=x(6);t2=x(7);delay2=x(8);
    Theta3=mod(x(9),2*pi);v3=x(10);t3=x(11);delay3=x(12);
    t_explosion1=t1+delay1;t_ex2=t2+delay2;t_ex3=t3+delay3;

    d1=[cos(Theta1),sin(Theta1),0];drop1=FY1+d1*v1*t1;
    d2=[cos(Theta2),sin(Theta2),0];drop2=FY2+d2*v2*t2;
    d3=[cos(Theta3),sin(Theta3),0];drop3=FY3+d3*v3*t3;
    C1=[drop1(1)+v1*d1(1)*delay1,drop1(2)+v1*d1(2)*delay1,drop1(3)-
0.5*g*delay1^2];
    C2=[drop2(1)+v2*d2(1)*delay2,drop2(2)+v2*d2(2)*delay2,drop2(3)-
0.5*g*delay2^2];
    C3=[drop3(1)+v3*d3(1)*delay3,drop3(2)+v3*d3(2)*delay3,drop3(3)-
0.5*g*delay3^2];
    if C1(3)<0,C1(3)=0;end
    if C2(3)<0,C2(3)=0;end
    if C3(3)<0,C3(3)=0;end

    t_explosion=[t_explosion1,t_ex2,t_ex3];
    expl_pos=[C1;C2;C3];
    t_start=max(min(t_explosion),0);
    t_end=min(max(t_explosion)+c_cover,t_boom);

    cover_time=0;intervals=[];
    if t_end<=t_start,return;end

    uM=(fake_target-M1)/norm(fake_target-M1);
```

```
shielding=false;cur_start=NaN;
t=t_start;
while t<=t_end+1e-12
    M_t=M1+uM*V_missile*t;
    covered=false;
    for j=1:3
        t_exj=t_explosion(j);
        if t<t_exj||t>t_exj+c_cover,continue;end
        C_t=expl_pos(j,:)+[0,0,-c_down*(t-t_exj)];
        v=true_target-M_t;
        vv=dot(v,v);
        if vv==0
            s_clamped=0;
        else
            s=dot(C_t-M_t,v)/vv;
            s_clamped=max(0,min(1,s));
        end
        closest=M_t+s_clamped*v;
        d=norm(C_t-closest);
        if d<=R
            covered=true;
            break;
        end
    end
    if covered
        cover_time=cover_time+dt;
        if ~shielding
            shielding=true;cur_start=t;
        end
    else
        if shielding
            shielding=false;
            intervals=[intervals;cur_start,t-dt];
```

```
                cur_start=NaN;
            end
        end
        t=t+dt;
    end
    if shielding
        intervals=[intervals;cur_start,t_end];
    end
end

%% 计算单枚弹的遮蔽区间
function
[cover_time,intervals]=single_bomb_interval(t_explosion_i,explosion_p
osition_i,idx_bomb,x,...

M1,V_missile,fake_target,true_target,FY1,FY2,FY3,g,R,c_down,c_cover,d
t,t_boom)

    Theta1=mod(x(1),2*pi);v1=x(2);t1=x(3);delay1=x(4);
    Theta2=mod(x(5),2*pi);v2=x(6);t2=x(7);delay2=x(8);
    Theta3=mod(x(9),2*pi);v3=x(10);t3=x(11);delay3=x(12);

    %当前单枚干扰弹有效存在区间
    if t_explosion_i>t_boom
        cover_time=0;intervals=[];return;
    end
    t_start=max(t_explosion_i,0);
    t_end=min(t_explosion_i+c_cover,t_boom);
    if t_end<=t_start
        cover_time=0;intervals=[];return;
    end
    %判断该干扰弹是否在该时刻遮蔽连线
    cover_time=0;intervals=[];
```

```
shielding=false;cur_start=NaN;
uM=(fake_target-M1)/norm(fake_target-M1);
t=t_start;
while t<=t_end+1e-12
    M_t=M1+uM*V_missile*t;
    %时间 t 时烟幕球心
    C_t=explosion_position_i+[0,0,-c_down*(t-t_explosion_i)];
    v=true_target-M_t;
    vv=dot(v,v);
    if vv==0
        s_clamped=0;
    else
        s=dot(C_t-M_t,v)/vv;
        s_clamped=max(0,min(1,s));
    end
    closest=M_t+s_clamped*v;
    dist=norm(C_t-closest);
    if dist<=R
        cover_time=cover_time+dt;
        if ~shielding
            shielding=true;cur_start=t;
        end
    else
        if shielding
            shielding=false;
            intervals=[intervals;cur_start,t-dt];
            cur_start=NaN;
        end
    end
    t=t+dt;
end
if shielding
    intervals=[intervals;cur_start,t_end];
```

```
    end
end
```